

Clase03

May 23, 2026

1 Ejercicios de programación

1.1 Tipos de datos

```
[1]: nombre = "Haydé tiene 25 años" # Esta es una variable de text "str"  
     print(nombre)
```

Haydé tiene 25 años

```
[2]: edad = 35 # Esta es una variable de numero "int"  
     print(edad)
```

35

```
[3]: estatura = 1.76 # Esta es una variable de numero "float"  
     print(estatura)
```

1.76

```
[4]: felicidad = True # Esta es una variable booleana "bool"  
     print(felicidad)
```

True

```
[5]: type(nombre)
```

```
[5]: str
```

```
[6]: type(edad)
```

```
[6]: int
```

```
[7]: type(estatura)
```

```
[7]: float
```

```
[8]: type(felicidad)
```

```
[8]: bool
```

```
[9]: comida_favorita = "pozole"
```

```
[10]: print(comida_favorita)
```

pozole

```
[11]: comida_favorita = "aguachile"
```

```
[12]: print(comida_favorita)
```

aguachile

```
[13]: # Operaciones con diferentes tipos de datos
      5 + 5
```

```
[13]: 10
```

```
[14]: "a" + "b"
```

```
[14]: 'ab'
```

```
[15]: suma = 5 + 5 #suma de enteros
      print (suma)
```

10

```
[16]: concat = "a" + "b" #concatenación de cadenas de text
      print(concat)
```

ab

```
[17]: prueba = comida_favorita + " " + str(estatura)
      print(prueba)
```

aguachile 1.76

```
[18]: type(estatura)
```

```
[18]: float
```

```
[19]: telefono = "3319753470"
```

```
[20]: telefono + 1
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[20], line 1
----> 1 telefono + 1

TypeError: can only concatenate str (not "int") to str
```

```
[21]: int(telefono) + 1
```

```
[21]: 3319753471
```

```
[22]: # Estructuras de control
# Condicionales      "==" igual, "!=" diferente, "<" menor que, ">" mayor que,
↳ <= "menor que o igual", >= "mayor que o igual"
felicidad = True

if felicidad != True:
    print("Toy bien felipe")
    resultado = 2+2
    if resultado > 3:
        print(resultado)
    else:
        print("No fue mayor que 3")
else:
    print("Toy bien sad")
```

Toy bien sad

```
[23]: # Bucles

prueba = 0
while prueba < 5:
    print("Voy en: " + str(prueba))
    prueba = prueba + 1
```

Voy en: 0

Voy en: 1

Voy en: 2

Voy en: 3

Voy en: 4

```
[25]: # Operaciones con cadenas de texto
version = 3.9
mensaje = "Estás usando python " + str(version)
print(mensaje)
```

Estás usando python 3.9

```
[26]: texto = "Hola mundo"
```

```
[27]: texto.upper()
```

```
[27]: 'HOLA MUNDO'
```

```
[29]: texto.replace("Hola", "Adios")
```

[29]: 'Adios mundo'

```
[61]: # Listas  
mi_lista = [1,2,3, "python", True]
```

```
[62]: mi_lista
```

[62]: [1, 2, 3, 'python', True]

```
[63]: mi_lista[0]
```

[63]: 1

```
[64]: len(mi_lista)
```

[64]: 5

```
[65]: # agregar valores  
mi_lista.append(4)  
mi_lista
```

[65]: [1, 2, 3, 'python', True, 4]

```
[66]: mi_lista.insert(2,[99,15])  
mi_lista
```

[66]: [1, 2, [99, 15], 3, 'python', True, 4]

```
[67]: mi_lista[2]
```

[67]: [99, 15]

```
[68]: mi_lista[2][0]
```

[68]: 99

```
[69]: mi_lista.remove(4)  
mi_lista
```

[69]: [1, 2, [99, 15], 3, 'python', True]

```
[70]: mi_lista.pop()  
mi_lista
```

[70]: [1, 2, [99, 15], 3, 'python']

```
[71]: mi_lista.sort()
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[71], line 1  
----> 1 mi_lista.sort()  
  
TypeError: '<' not supported between instances of 'list' and 'int'
```

```
[73]: lista2 = [3,5,1,7,9]  
      lista2.sort()  
      lista2
```

```
[73]: [1, 3, 5, 7, 9]
```

```
[74]: lista3 = ["b","a","c"]  
      lista3.sort()  
      lista3
```

```
[74]: ['a', 'b', 'c']
```

```
[77]: lista2.reverse()  
      lista2
```

```
[77]: [9, 7, 5, 3, 1]
```

```
[78]: lista2.append(9)  
      lista2
```

```
[78]: [9, 7, 5, 3, 1, 9]
```

```
[79]: lista2.count(9)
```

```
[79]: 2
```

```
[80]: lista2.pop(0)  
      lista2
```

```
[80]: [7, 5, 3, 1, 9]
```

```
[81]: lista2.pop(3)  
      lista2
```

```
[81]: [7, 5, 3, 9]
```

```
[85]: resultado = "Hola,mi,nombre,es,Hayde".split(",")
```

```
[86]: resultado
```

```
[86]: ['Hola', 'mi', 'nombre', 'es', 'Hayde']
```

```
[87]: # Tuplas  
mi_tupla = (1,2,3,"Python")
```

```
[88]: mi_tupla
```

```
[88]: (1, 2, 3, 'Python')
```

```
[89]: mi_tupla.append(1)
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In[89], line 1  
----> 1 mi_tupla.append(1)  
  
AttributeError: 'tuple' object has no attribute 'append'
```

```
[90]: mi_tupla.count(2)
```

```
[90]: 1
```

```
[92]: mi_tupla.index("Python")
```

```
[92]: 3
```

```
[93]: lista2
```

```
[93]: [7, 5, 3, 9]
```

```
[100]: for i in lista2:  
        print(i)
```

```
7  
5  
3  
9
```

```
[101]: for b in mi_tupla:  
        print(b)
```

```
1  
2  
3  
Python
```

```
[102]: list(mi_tupla)
```

[102]: [1, 2, 3, 'Python']

```
[103]: tuple(lista2)
```

[103]: (7, 5, 3, 9)

```
[108]: # Diccionarios
```

```
mi_dict = {"nombre": "Python", "version": "3.9", "activo": True, "valores": [1, 2, 3]}
mi_dict
```

[108]: {'nombre': 'Python', 'version': '3.9', 'activo': True, 'valores': [1, 2, 3]}

```
[109]: mi_dict["nombre"]
```

[109]: 'Python'

```
[110]: mi_dict["version"]
```

[110]: '3.9'

```
[111]: mi_dict["activo"]
```

[111]: True

```
[112]: mi_dict["valores"]
```

[112]: [1, 2, 3]

```
[113]: mi_dict["valores"][0]
```

[113]: 1

```
[115]: mi_dict["nombre"] = "Py"
mi_dict
```

[115]: {'nombre': 'Py',
 'version': '3.9',
 'activo': True,
 'valores': [1, 2, 3],
 'extra': 1234}

```
[121]: mi_dict.keys() #ver todas mis claves
```

[121]: dict_keys(['nombre', 'version', 'activo', 'valores', 'extra'])

```
[122]: mi_dict.values() #ver todos mis valores
```

[122]: dict_values(['Py', '3.9', True, [1, 2, 3], 1234])

```
[123]: for key in mi_dict.keys():  
        print( key + ":" + str(mi_dict[key]))
```

```
nombre:Py  
version:3.9  
activo:True  
valores:[1, 2, 3]  
extra:1234
```

```
[125]: for values in mi_dict.values():  
        print(str(values))
```

```
Py  
3.9  
True  
[1, 2, 3]  
1234
```

```
[127]: mi_dict.items()
```

```
[127]: dict_items([('nombre', 'Py'), ('version', '3.9'), ('activo', True), ('valores',  
[1, 2, 3]), ('extra', 1234)])
```

```
[128]: for key, value in mi_dict.items():  
        print(key + ":" + str(value))
```

```
nombre:Py  
version:3.9  
activo:True  
valores:[1, 2, 3]  
extra:1234
```

```
[130]: mi_dict.pop("activo")
```

```
[130]: True
```

```
[132]: mi_dict
```

```
[132]: {'nombre': 'Py', 'version': '3.9', 'valores': [1, 2, 3], 'extra': 1234}
```

```
[133]: mi_dict.clear()
```

```
[134]: mi_dict
```

```
[134]: {}
```

```
[135]: mi_dict.get("version", "no existe")
```

```
[135]: 'no existe'
```



```
[138]: #Slicing  
frase = "Hola como estas"  
frase[0]
```

```
[138]: 'H'
```

```
[139]: frase[5]
```

```
[139]: 'c'
```

```
[140]: palabra = frase[0:4]  
palabra
```

```
[140]: 'Hola'
```

```
[143]: frase[5:10] + frase[0:5] + frase[10:]
```

```
[143]: 'como Hola estas'
```

```
[144]: frase[10:]
```

```
[144]: 'estas'
```

```
[145]: frase[:5]
```

```
[145]: 'Hola '
```

```
[146]: # Funciones  
def saludar():  
    print("Hola")
```

```
[147]: saludar()
```

```
Hola
```

```
[148]: def saludar2(nombre):  
        print("Hola " + nombre)
```

```
[150]: saludar2("Brenda")
```

```
Hola Brenda
```

```
[151]: saludar2("HErminio")
```

```
Hola HErminio
```

```
[152]: def sumar(a,b):  
        return a + b
```

```
[153]: sumar(1,2)
```

[153]: 3

```
[154]: resultado = sumar(1,2)
```

```
[155]: resultado
```

[155]: 3

```
[160]: # 2. Escribir una función que reciba una tupla con n cantidad de nombres, y
      ↪ para cada nombre imprima
      # el mensaje Estimado,*inserte nombre aquí* vote por mí.
```

```
def voto(tupla):
    for elemento in tupla:
        print("Estimado "+str(elemento)+" vote por mi")
```

```
[161]: voto(mi_tupla)
```

```
Estimado 1 vote por mi
Estimado 2 vote por mi
Estimado 3 vote por mi
Estimado Python vote por mi
```

```
[162]: tupli = (1,2)
      tupli2 = (2,8)
      tupli3 = (5,7)
```

```
[166]: if tupli[0] in tupli3 or tupli[1] in tupli3:
      print("si esta")
      else:
          print("no esta")
```

no esta

```
[167]: if tupli[1] in tupli2 and tupli[1] > 3:
      print("Si cumple todas las condiciones")
      else:
          print("No cumple todas las condiciones")
```

No cumple todas las condiciones

```
[ ]:
```